GUIA VISUAL 064

EXPRESSÕES REGULARES

064
GUIAS LINUX

*O servidor estava sob força bruta e o analista rodou `grep 10.0.0.1 /var/log/auth.log`. Vieram as linhas certas — e uma com `id=10a0b0c1`, que não tem endereço nenhum. O ponto que ele digitou querendo dizer **ponto** significa, para a ferramenta, **qualquer caractere**. Na tentativa seguinte, `grep 'Failed+'` devolveu zero linhas — sem erro, sem aviso: o `grep` sem `-E` não conhece o `+`. Um padrão descreve **forma**, e há **três dialetos** dessa notação na mesma máquina.*

ANATOMIA DE UM PADRÃO REAL — ENDEREÇO IPV4

```
^ ( [0-9] {1,3} \. ) {3} [0-9] {1,3} $
```

`^ $` âncora
Condição sobre a **posição**: não consome caractere. Daí `^$` ser "linha vazia".

`( )` grupo
Delimita o item para o quantificador **e captura** o trecho como `\1`.

`[ ]` classe
Um caractere dentre os listados. `[^0-9]` nega; `[.]` desarma o ponto.

`{n,m}` quantifica
Repete só o item **imediatamente anterior**. Em `ab*`, o `*` governa só o `b`.

`\.` literal
A barra **tira** o poder do símbolo. `\.` é ponto de verdade.

Lido da esquerda para a direita: *da borda esquerda da linha, três repetições de "um a três dígitos seguidos de ponto", depois mais um a três dígitos, e a borda direita.* Note o que ele **não** diz: nada impede `999.999.999.999`. **Expressão regular reconhece forma; validar valor é outro problema** — trabalho de `awk` ou de uma linguagem. Quando o padrão passa de três linhas ou precisa *contar*, troque de ferramenta: HTML, JSON e parênteses balanceados **não são linguagens regulares** e nenhum padrão resolve o caso geral. E não confunda com o **globbing**: o glob é do **shell** e opera sobre nomes de arquivo (`*` = qualquer sequência); a expressão regular é da **ferramenta** e opera sobre linhas (`*` = zero ou mais do item anterior).

OS TRÊS SABORES QUE CONVIVEM NA MESMA MÁQUINA

<table>
<tr><th>Construto</th><th>BRE
grep · sed · expr</th><th>ERE
grep -E · sed -E · awk</th><th>PCRE
grep -P · perl · python</th></tr>
<tr><td>uma ou mais</td><td><code>a\+</code></td><td><code>a+</code></td><td><code>a+</code></td></tr>
<tr><td>opcional</td><td><code>a\?</code></td><td><code>a?</code></td><td><code>a?</code></td></tr>
<tr><td>repetição contada</td><td><code>a\{2,5\}</code></td><td><code>a{2,5}</code></td><td><code>a{2,5}</code></td></tr>
<tr><td>grupo</td><td><code>\(ab\)</code></td><td><code>(ab)</code></td><td><code>(ab)</code></td></tr>
<tr><td>alternância</td><td><code>a\|b</code></td><td><code>a|b</code></td><td><code>a|b</code></td></tr>
<tr><td>dígito</td><td><code>[[:digit:]]</code></td><td><code>[[:digit:]]</code></td><td><code>\d</code></td></tr>
<tr><td>preguiçoso · olhar adiante</td><td>não existe</td><td>não existe</td><td><code>.*? (?=…)</code></td></tr>
</table>

A regra que resolve quase tudo: em **BRE** o construto avançado exige barra invertida e o símbolo cru é literal; em **ERE** é o contrário — a barra só serve para *tirar* o poder. **Na dúvida, `grep -E`**: é o mesmo padrão que você escreveria em `awk`, `sed -E` ou em qualquer linguagem moderna. O `egrep` e o `fgrep` ainda rodam, mas imprimem aviso de depreciação no Kali atual. E há uma quarta variante que não é expressão regular nenhuma: `grep -F` desliga a interpretação inteira, busca texto literal e é a mais rápida.

`grep ^root /etc/passwd`
Padrão vai entre aspas simples, sempre. Sem elas o shell processa `*`, `?`, `[…]`, `$` e `\` **antes** da ferramenta ver o padrão — e o que chega ao `grep` já não é o que você escreveu.

`grep -E '^(erro|falha)'`
A alternância tem a menor precedência de todas. `^erro|falha` significa "começa com erro" **ou** "contém falha" em qualquer posição — quase nunca o que se quer. Agrupe.

O `.*` é guloso e atravessa delimitador. Numa linha com três campos entre aspas, `grep -oE '".*"'` devolve **uma cadeia gigante**, da primeira à última aspa. Negue em vez de generalizar: **`"[^"]*"`** — em PCRE existe também o preguiçoso **`".*?"`**. **Faixa depende da localidade:** `[a-z]` muda de resultado conforme `LC_ALL`; as **classes POSIX nomeadas** são estáveis — `[[:digit:]]`, `[[:alpha:]]`, `[[:alnum:]]`, `[[:space:]]`, `[[:upper:]]`, `[[:punct:]]`, `[[:xdigit:]]`. Dentro dos colchetes as regras mudam: o `^` só nega na **primeira** posição, o `]` só é literal **logo no início** e o `-` só é literal no **começo ou no fim**. Bordas de palavra `\<pai\>` e `\bpai\b` impedem que `10.0.0.1` case dentro de `10.0.0.14`.

Retrocesso catastrófico (ReDoS) — não rode em produção. O motor POSIX (`grep -E`, `sed`, `awk`) usa autômato finito: casa em tempo proporcional ao tamanho do texto e **não retrocede**. O motor PCRE (`grep -P`, linguagens) retrocede — e um quantificador aninhado explode: `echo "$(printf 'a%.0s' {1..40})" | grep -qP '(a+)+b'` prende um núcleo por minutos, porque o motor tenta um número **exponencial** de divisões antes de desistir. É por isso que `(a|ab)` casa `ab` em POSIX (mais longo) e `a` em PCRE (primeira alternativa que fecha). Defesas: prefira `-E` a `-P` quando os dois servirem; em PCRE, nada de quantificador dentro de grupo quantificado, e classe negada no lugar de `.*`.

RESUMO RÁPIDO

NO KALI